

LAPORAN TUGAS BESAR I STRATEGI ALGORITMA

Laporan ini disusun untuk memenuhi Tugas Mata Kuliah :

Strategi Algoritma



Disusun Oleh :

Nama : 1. Erhan Kurniawan /124140217

2. Crisman Panorangi Turnip /124140118

Program Studi : Teknik Informatika

Mata Kuliah : Strategi Algoritma

Dosen Pengampu : Imam Eko Wicaksono, S.Si., M.Si.

Kelas : RA

**PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
TAHUN 2026**

BAB I: Deskripsi Tugas

1.1 Latar Belakang

Robocode Tank Royale adalah permainan pemrograman yang mengharuskan peserta membuat bot tank untuk bertarung di dalam arena. Bot tidak dikendalikan secara manual saat pertandingan berlangsung. Semua keputusan bot harus ditulis dalam program, mulai dari mencari musuh, bergerak, menembak, menghindari dinding, sampai merespons serangan lawan.

Pada tugas besar ini, strategi utama yang digunakan adalah algoritma greedy. Pendekatan greedy cocok digunakan karena Robocode Tank Royale berjalan dalam satuan turn. Pada setiap turn, bot harus cepat memilih aksi yang dianggap paling menguntungkan berdasarkan informasi yang tersedia saat itu.

Dengan kondisi permainan yang terus berubah, bot tidak dapat hanya menjalankan pola gerakan tetap. Bot perlu menilai keadaan arena secara berulang, misalnya apakah musuh berada terlalu dekat, apakah energi sendiri masih cukup untuk menembak, atau apakah posisi bot sudah terlalu dekat dengan dinding. Oleh karena itu, tugas ini tidak hanya berfokus pada membuat bot yang dapat berjalan, tetapi juga pada bagaimana bot mengambil keputusan sederhana yang masuk akal dalam waktu singkat.

1.2 Tujuan Pengerjaan

Tujuan dari pengerjaan tugas ini adalah:

1. Membuat bot Robocode Tank Royale yang dapat dijalankan pada engine.
2. Menerapkan konsep algoritma greedy ke dalam pengambilan keputusan bot.
3. Membuat empat alternatif strategi greedy dengan heuristic yang berbeda.
4. Membandingkan alternatif strategi tersebut dan memilih strategi utama.
5. Melakukan pengujian 1 vs 1 antara bot utama dan bot pembanding.

Melalui tujuan tersebut, hasil akhir yang diharapkan bukan hanya bot yang dapat mengikuti pertandingan, tetapi juga bot yang memiliki alasan strategis pada setiap keputusan utamanya. Setiap alternatif bot dibuat untuk menunjukkan bahwa greedy dapat diterapkan dengan sudut pandang berbeda, seperti mengejar damage, menjaga jarak, mencari musuh lemah, atau menyeimbangkan serangan dan pertahanan.

1.3 Ruang Lingkup Program

Proyek TubesStima memiliki satu bot utama dan tiga bot alternatif. Bot utama yang dipilih adalah **Kurayami**, sedangkan bot alternatif yang digunakan sebagai pembanding adalah **Satsugi**, **Shinji**, dan **Tatsuya**.

Keempat bot tersebut dibuat dengan dasar yang sama, yaitu memanfaatkan informasi dari radar dan event permainan. Perbedaannya terletak pada heuristic yang dipakai untuk menentukan keputusan. Dengan adanya beberapa bot, strategi utama dapat dibandingkan dengan pendekatan lain sehingga pemilihan bot utama memiliki dasar yang lebih jelas.

Ringkasan peran masing-masing bot:

Bot	Peran	Karakter Strategi
Kurayami	Bot utama	Seimbang antara menyerang, menjaga jarak, dan bertahan
Satsugi	Bot alternatif	Agresif, mengejar target, dan menekan lawan
Shinji	Bot alternatif	Defensif, menjaga jarak aman, dan menghindari risiko
Tatsuya	Bot alternatif	Mengejar musuh dengan energi paling rendah

1.4 Gambaran Aturan Robocode Tank Royale

Pertandingan berjalan dalam beberapa round. Setiap round terdiri dari banyak turn. Pada setiap turn, bot dapat memberi perintah untuk bergerak, memutar badan, memutar radar, memutar gun, atau menembak.

Aturan permainan sangat berpengaruh terhadap desain strategi. Misalnya, bot tidak boleh terlalu sering menembak dengan power besar karena energi akan cepat habis. Bot juga tidak boleh terlalu pasif karena skor banyak diperoleh dari damage. Selain itu, radar harus terus dimanfaatkan agar bot tidak kehilangan informasi posisi musuh.

Beberapa aturan penting yang memengaruhi strategi bot adalah:

1. **Energi** digunakan sebagai nyawa sekaligus sumber daya untuk menembak.
2. **Peluru** dengan power besar menghasilkan damage besar, tetapi lebih lambat dan lebih mahal.
3. **Gun heat** membuat bot tidak bisa menembak terus-menerus.
4. **Radar** harus aktif bergerak agar bot dapat menemukan musuh.

5. **Dinding** harus dihindari karena tabrakan dapat mengurangi energi.
6. **Skor** diperoleh dari damage, survival, bonus kill, dan ramming.

1.5 Cara Menjalankan Bot

Bot dapat dijalankan dari folder masing-masing bot. Contoh menjalankan bot utama:

```
bash  
cd Tubes_Tantrump/src/main-bot/Kurayami  
dotnet build  
dotnet run --no-build
```

Pada Windows, bot juga dapat dijalankan melalui file:

```
text  
Kurayami.cmd
```

File `.cmd` dan `.sh` disediakan agar proses menjalankan bot lebih mudah. Pada mode pengembangan, script dapat membersihkan hasil build lama, melakukan build ulang, lalu menjalankan bot. Cara ini membantu memastikan kode terbaru yang dijalankan adalah kode yang sedang diuji.

BAB II: Landasan Teori

2.1 Konsep Dasar Algoritma Greedy

Algoritma greedy adalah algoritma yang membangun solusi dengan memilih keputusan terbaik pada setiap langkah. Keputusan tersebut dipilih berdasarkan keuntungan lokal, yaitu keuntungan yang terlihat paling baik pada kondisi saat itu.

Greedy tidak selalu menjamin solusi global terbaik. Namun, greedy memiliki keunggulan karena sederhana, cepat, dan mudah diterapkan pada masalah yang membutuhkan keputusan real-time. Hal ini sesuai dengan Robocode Tank Royale, karena bot harus mengambil keputusan dalam waktu singkat pada setiap turn.

Dalam konteks permainan, keputusan lokal dapat berupa memilih target paling dekat, menembak dengan power tertentu, atau bergerak menjauh dari dinding. Keputusan tersebut belum tentu menjadi keputusan terbaik untuk seluruh pertandingan, tetapi dapat membantu bot bertahan dan menyerang secara konsisten dari turn ke turn.

2.2 Elemen Algoritma Greedy

Dalam algoritma greedy, terdapat beberapa elemen penting:

Elemen	Penjelasan
Himpunan kandidat	Kumpulan pilihan yang tersedia
Himpunan solusi	Pilihan yang sudah diambil sebagai solusi sementara
Fungsi seleksi	Aturan untuk memilih kandidat terbaik
Fungsi kelayakan	Aturan untuk memastikan pilihan dapat dilakukan
Fungsi solusi	Kondisi yang menyatakan solusi sudah terbentuk
Fungsi objektif	Tujuan yang ingin dimaksimalkan atau diminimalkan

Elemen-elemen tersebut membantu menjelaskan strategi bot secara lebih sistematis. Tanpa pemetaan ini, keputusan bot hanya terlihat seperti kumpulan aturan. Dengan pemetaan greedy, setiap aturan dapat dihubungkan dengan tujuan tertentu, misalnya memaksimalkan skor, mengurangi risiko, atau memilih target yang paling menguntungkan.

2.3 Greedy dalam Konteks Robocode

Pada Robocode Tank Royale, keputusan greedy dapat muncul dalam beberapa bentuk. Bot dapat memilih target yang paling dekat, target dengan energi paling rendah, arah gerak yang paling aman, atau power tembakan yang paling sesuai. Setiap pilihan tersebut adalah keputusan lokal yang diambil berdasarkan data pada turn berjalan.

Data yang digunakan oleh bot berasal dari kondisi internal bot dan event dari engine. Kondisi internal mencakup energi, posisi, arah, gun heat, dan posisi terhadap dinding. Event dari engine memberikan informasi seperti posisi musuh ketika terdeteksi, kondisi ketika terkena peluru, atau ketika bot bertabrakan. Semua informasi ini kemudian digunakan sebagai dasar heuristic.

Contoh keputusan greedy pada bot:

1. Jika musuh terlalu dekat, bot memilih menjauh.
2. Jika target berenergi rendah, bot dapat meningkatkan prioritas serangan.
3. Jika bot dekat dinding, bot memilih bergerak ke tengah arena.
4. Jika energi sendiri rendah, bot mengurangi power tembakan.
5. Jika target jauh, bot menembak dengan power lebih kecil agar energi tidak boros.

2.4 Alur Kerja Bot

Secara umum, bot bekerja melalui kombinasi loop utama dan event. Loop utama berjalan selama pertandingan berlangsung, sedangkan event dipanggil ketika terjadi kejadian tertentu.

Loop utama bertugas menjaga bot tetap aktif, misalnya terus memutar radar dan menjalankan movement. Sementara itu, event membuat bot dapat bereaksi terhadap kondisi mendadak. Kombinasi keduanya penting karena bot harus memiliki perilaku umum, tetapi tetap bisa menyesuaikan diri ketika situasi berubah.

Alur kerja bot:

1. Bot mengaktifkan radar untuk mencari musuh.
2. Ketika musuh terdeteksi, data musuh disimpan.
3. Bot mengevaluasi target berdasarkan heuristic greedy.
4. Bot menentukan gerakan berdasarkan jarak, posisi, dan risiko dinding.
5. Bot mengarahkan gun ke target atau posisi prediksi target.
6. Bot menembak jika gun siap, energi cukup, dan arah tembakan layak.
7. Bot mengubah perilaku ketika terkena peluru, menabrak dinding, menabrak bot lain, atau target mati.

BAB III: Aplikasi Strategi Greedy

3.1 Formulasi Masalah

Masalah utama pada Robocode Tank Royale adalah memilih aksi terbaik pada setiap turn agar bot memperoleh skor setinggi mungkin. Aksi yang dipilih tidak hanya harus menghasilkan damage, tetapi juga harus menjaga bot agar tetap hidup.

Jika bot hanya fokus menyerang, bot dapat kehilangan banyak energi karena terlalu sering menembak atau terlalu dekat dengan musuh. Sebaliknya, jika bot hanya fokus bertahan, bot sulit memperoleh skor dari damage. Oleh karena itu, strategi yang baik perlu menyeimbangkan peluang menyerang dan peluang bertahan.

Fungsi objektif umum dapat ditulis secara sederhana sebagai berikut:

text

Maksimalkan skor =

- bullet damage**
- + bonus kill**
- + survival score**
- + last survival bonus**
- + ram damage**
- + ram bonus**

Karena skor dipengaruhi oleh serangan dan pertahanan, strategi bot tidak boleh hanya fokus pada menembak. Bot juga harus menjaga jarak, menghindari dinding, dan menghemat energi.

Fungsi objektif ini menjadi dasar dalam merancang heuristic. Pada implementasi Kurayami, skor tidak dihitung secara langsung setiap turn, tetapi diperkirakan melalui indikator yang lebih sederhana, seperti jarak ke musuh, energi musuh, energi bot sendiri, dan kualitas tembakan. Pendekatan ini membuat keputusan tetap cepat.

3.2 Pemetaan ke Elemen Greedy

Pemetaan persoalan Robocode Tank Royale ke elemen greedy adalah sebagai berikut:

Elemen Greedy	Penerapan pada Bot
Himpunan kandidat	Musuh yang terdeteksi, arah gerak, power peluru, aksi radar

Himpunan solusi	Target yang dipilih, gerakan yang dilakukan, tembakan yang dilepaskan
Fungsi seleksi	Memilih target atau aksi dengan nilai heuristic terbaik
Fungsi kelayakan	Mengecek energi, gun heat, validitas target, dan risiko dinding
Fungsi solusi	Bot berhasil menentukan aksi untuk turn saat ini
Fungsi objektif	Memaksimalkan skor dan survival dengan sumber daya energi terbatas

Pemetaan ini menunjukkan bahwa setiap turn dapat dianggap sebagai satu langkah pembentukan solusi. Pada satu turn, bot memilih satu atau beberapa aksi yang layak, lalu menjalankannya. Pada turn berikutnya, bot kembali mengevaluasi kondisi terbaru dan memilih aksi berikutnya.

3.3 Empat Alternatif Strategi Greedy

Pada proyek ini, empat bot dirancang dengan heuristic yang berbeda. Setiap bot mewakili pendekatan greedy yang berbeda terhadap persoalan yang sama.

Perbedaan heuristic ini penting karena tidak ada satu strategi yang selalu unggul pada semua kondisi. Bot agresif dapat unggul ketika lawan lemah, tetapi rentan terhadap lawan yang menjaga jarak. Bot defensif lebih aman, tetapi kadang kurang cepat menghasilkan skor. Karena itu, keempat alternatif dianalisis sebelum menentukan strategi utama.

3.3.1 Kurayami - Greedy Seimbang Berdasarkan Target Score

Kurayami memilih target berdasarkan gabungan beberapa faktor. Musuh yang lebih dekat diberi nilai lebih tinggi, musuh dengan energi rendah mendapat bonus, dan target yang lama tidak terlihat mendapat penalti.

Pendekatan ini membuat Kurayami tidak terlalu mudah berpindah target tanpa alasan, tetapi tetap fleksibel jika ada target yang lebih menguntungkan. Strategi ini juga membuat Kurayami lebih stabil dalam kondisi banyak musuh karena target dipilih berdasarkan nilai, bukan hanya berdasarkan musuh yang pertama terlihat.

Heuristic utama:

text

target_score =
nilai_jarak
+ bonus_energi_rendah
- penalti_target_lama_tidak_terlihat

Karakter strategi:

1. Mengunci radar pada target yang dipilih.
2. Menjaga jarak aman dari target.
3. Menghindari dinding.
4. Mengatur power tembakan berdasarkan jarak, energi, dan akurasi.
5. Mengubah arah ketika terkena peluru atau menabrak dinding.

Kelayakan aksi pada Kurayami dipengaruhi oleh beberapa kondisi, yaitu target belum terlalu lama hilang, bot tidak berada terlalu dekat dengan dinding, energi cukup untuk menembak, dan gun sudah cukup mengarah ke target.

3.3.2 Satsugi - Greedy Agresif Mengejar Target

Satsugi menggunakan pendekatan yang lebih agresif. Bot ini memilih aksi yang membuatnya cepat mendekat ke target. Saat jarak sudah dekat, Satsugi menggunakan tembakan dengan power besar agar damage yang dihasilkan tinggi.

Gaya permainan Satsugi lebih cocok untuk menekan lawan dan memaksa pertarungan jarak dekat. Dalam jarak dekat, peluang peluru mengenai target lebih besar sehingga penggunaan power tinggi menjadi lebih masuk akal. Namun, pendekatan ini membutuhkan kontrol dinding yang baik agar bot tidak kehilangan momentum.

Heuristic utama:

text

pilih aksi yang memperkecil jarak ke target
dan meningkatkan peluang damage jarak dekat

Karakter strategi:

1. Bergerak cepat menuju target.
2. Menggunakan prediksi posisi target sebelum menembak.
3. Menggunakan fire power besar pada jarak dekat.
4. Tetap menghindari dinding agar tidak kehilangan tempo.

Strategi ini cocok untuk memberi tekanan besar pada lawan, tetapi lebih berisiko karena bot dapat menjadi mudah ditembak saat terlalu agresif mendekati musuh.

3.3.3 Shinji - Greedy Bertahan dengan Jarak Aman

Shinji lebih mengutamakan keselamatan posisi. Bot ini memilih aksi yang menjaga jarak ideal dari musuh. Jika musuh terlalu dekat, Shinji bergerak diagonal menjauh. Jika jarak sudah aman, bot bergerak melingkar untuk tetap sulit ditembak.

Strategi Shinji menunjukkan bahwa greedy tidak selalu berarti menyerang secepat mungkin. Pada kasus ini, keputusan terbaik pada suatu turn sering kali adalah mengurangi risiko terlebih dahulu. Dengan menjaga jarak, Shinji berusaha mempertahankan survival score sambil tetap mencari kesempatan menembak.

Heuristic utama:

text

**pilih aksi dengan risiko paling kecil
berdasarkan jarak musuh dan posisi terhadap dinding**

Karakter strategi:

1. Menjaga jarak ideal dari target.
2. Menghindari ramming.
3. Mengurangi power tembakan saat energi rendah.
4. Mengutamakan survival dibanding serangan jarak dekat.

Strategi ini baik untuk menghadapi bot agresif, tetapi damage yang dihasilkan dapat lebih kecil karena bot tidak selalu mengambil risiko untuk menyerang.

3.3.4 Tatsuya - Greedy Mengejar Energi Terendah

Tatsuya memilih musuh dengan energi paling rendah sebagai target. Tujuannya adalah memperbesar peluang eliminasi dan memperoleh bonus kill.

Strategi ini cukup langsung dan mudah dipahami. Jika ada musuh yang energinya rendah, maka musuh tersebut dianggap sebagai target yang paling menguntungkan karena lebih mudah dikalahkan. Fokus utama Tatsuya adalah menyelesaikan target yang sudah lemah sebelum musuh tersebut sempat menjauh atau diserang bot lain.

Heuristic utama:

text

**pilih musuh dengan energi paling rendah
sebagai target utama**

Karakter strategi:

1. Mengunci target dengan energi paling rendah.
2. Mengejar target sampai target hilang atau mati.
3. Menggunakan fire power lebih besar ketika target berpotensi dieliminasi.
4. Melakukan prediksi posisi target secara linear.

Strategi ini efektif jika target lemah dapat dikejar. Namun, strategi ini dapat berbahaya jika bot terlalu fokus pada target lemah dan mengabaikan musuh lain yang lebih dekat.

3.4 Analisis Efisiensi

Keempat strategi memiliki beban komputasi yang ringan. Sebagian besar keputusan hanya menggunakan perbandingan angka, perhitungan jarak, sudut, dan operasi trigonometri sederhana.

Efisiensi menjadi penting karena bot harus mengirim keputusan dalam batas waktu turn. Strategi yang terlalu berat dapat membuat bot terlambat merespons. Dengan perhitungan yang sederhana, keempat bot tetap dapat mengambil keputusan secara cepat meskipun pertandingan berlangsung dinamis.

Bot	Operasi Utama	Kompleksitas Waktu	Kompleksitas Memori
Kurayami	Target score, radar lock, prediksi linear, movement lateral	$O(1)$ per scan	$O(1)$
Satsugi	Pengejaran agresif dan prediksi posisi terbatas	$O(1)$, karena batas prediksi tetap	$O(1)$
Shinji	Evaluasi jarak aman dan arah gerak	$O(1)$ per turn	$O(1)$
Tatsuya	Seleksi energi terendah dan prediksi linear	$O(1)$ per scan	$O(1)$

Pada implementasi ini, setiap bot hanya menyimpan satu target utama atau satu snapshot musuh. Oleh karena itu, penggunaan memori tetap kecil dan tidak bertambah besar selama pertandingan.

3.5 Analisis Efektivitas

Perbandingan efektivitas tiap strategi:

Strategi	Efektif Ketika	Kurang Efektif Ketika
Kurayami	Target terlihat stabil dan bot berada pada jarak menengah	Target sangat lincah atau bot terpojok dinding
Satsugi	Lawan lambat, pasif, atau sulit menjaga jarak	Lawan memiliki aim kuat dan mampu menjaga jarak
Shinji	Menghadapi bot agresif atau rammer	Perlu mengejar skor damage dengan cepat
Tatsuya	Ada musuh berenergi rendah yang mudah dikejar	Ancaman terbesar bukan target dengan energi terendah

Efektivitas setiap strategi sangat bergantung pada karakter lawan. Dalam 1 vs 1, strategi yang menjaga jarak dan akurasi cenderung lebih stabil. Dalam kondisi banyak bot, strategi mengejar target lemah dapat memberi keuntungan, tetapi juga dapat membuat bot tidak memperhatikan ancaman lain di arena.

3.6 Strategi yang Dipilih

Strategi yang dipilih sebagai strategi utama adalah **Kurayami**. Pemilihan ini didasarkan pada keseimbangan strategi. Kurayami tidak hanya mengandalkan satu heuristic, tetapi menggabungkan beberapa pertimbangan penting: jarak, energi musuh, usia informasi radar, energi sendiri, akurasi tembakan, dan risiko dinding.

Kurayami dipilih karena pendekatannya paling sesuai dengan tujuan umum tugas, yaitu memperoleh skor sambil tetap bertahan. Strategi ini tidak terlalu ekstrem seperti Satsugi, tidak terlalu defensif seperti Shinji, dan tidak terlalu fokus pada satu faktor seperti Tatsuya. Dengan demikian, Kurayami menjadi pilihan yang lebih aman untuk dijadikan bot utama.

Alasan pemilihan Kurayami:

1. Lebih stabil dibanding strategi yang terlalu agresif.
2. Lebih aktif menyerang dibanding strategi yang terlalu defensif.
3. Memiliki target memory sehingga tidak mudah kehilangan fokus.
4. Memiliki adaptive fire power agar energi tidak cepat habis.
5. Memiliki wall avoidance dan movement lateral untuk meningkatkan survival.

BAB IV: Implementasi dan Pengujian

4.1 Struktur Program

Setiap bot berada pada folder masing-masing dan memiliki file utama:

Bot	File Implementasi	File Metadata
Kurayami	`src/main-bot/Kurayami/Kurayami.cs`	`Kurayami.json`
Satsugi	`src/alternative-bots/Satsugi/Satsugi.cs`	`Satsugi.json`
Shinji	`src/alternative-bots/Shinji/Shinji.cs`	`Shinji.json`
Tatsuya	`src/alternative-bots/Tatsuya/Tatsuya.cs`	`Tatsuya.json`

File `.cs` berisi logika bot, sedangkan file `.json` berisi metadata bot seperti nama, versi, author, deskripsi, game type, platform, dan bahasa pemrograman.

Selain itu, setiap bot juga memiliki file script seperti `.cmd` dan `.sh` untuk membantu proses build dan run. Struktur ini membuat setiap bot dapat dijalankan secara terpisah sehingga pengujian antarbot lebih mudah dilakukan.

4.2 Pseudocode Empat Alternatif

4.2.1 Kurayami

text

BEGIN Kurayami

Atur warna, kecepatan, radar, dan gun

target <- kosong

WHILE bot berjalan DO

IF target sudah lama tidak terlihat THEN

target <- kosong

ENDIF

IF target kosong THEN

```

    putar radar untuk mencari musuh
    bergerak patroli dengan aman
ELSE
    kunci radar ke target

    IF target terlalu dekat THEN
        bergerak menjauh dari target
    ELSE IF bot dekat dinding THEN
        bergerak menuju tengah arena
    ELSE
        bergerak menyamping terhadap target
    ENDIF
ENDIF

Go()
ENDWHILE

EVENT OnScannedBot(e)
    hitung skor kandidat target

    IF kandidat lebih baik dari target saat ini THEN
        simpan data target
    ENDIF

    prediksi posisi target
    pilih power tembakan

    IF gun siap AND aim cukup tepat AND energi cukup THEN
        tembak

```

```
ENDIF
END_EVENT
END
```

4.2.2 Satsugi

```
text
BEGIN Satsugi
    target <- kosong

    WHILE bot berjalan DO
        IF target hilang THEN
            radar mencari musuh
            bot bergerak mencari target
        ELSE
            arahkan radar ke target
            prediksi posisi target
            bergerak cepat mendekati target

            IF gun siap dan arah tembak layak THEN
                tembak dengan power sesuai jarak
            ENDIF
        ENDIF
    ENDWHILE

    Go()
END
```

4.2.3 Shinji

text

BEGIN Shinji

WHILE bot berjalan DO

IF tidak ada target THEN

 lakukan patroli sambil scanning

ELSE IF bot dekat dinding THEN

 bergerak ke tengah arena

ELSE IF target terlalu dekat THEN

 bergerak diagonal menjauh

ELSE IF target pada jarak ideal THEN

 bergerak melingkar

ELSE

 mendekati target secara miring

ENDIF

IF target valid AND gun siap AND lock stabil THEN

 pilih power tembakan

 tembak jika aim sudah tepat

ENDIF

Go()

ENDWHILE

END

4.2.4 Tatsuya

text

BEGIN Tatsuya

locked <- false

WHILE bot berjalan DO

IF target terkunci terlalu lama tidak terlihat THEN

hapus lock

ENDIF

IF locked THEN

kejar target

prediksi posisi target

arahkan gun dan radar

IF gun siap dan energi cukup THEN

tembak

ENDIF

ELSE

putar radar mencari target

ENDIF

Go()

ENDWHILE

EVENT OnScannedBot(e)

IF belum ada target OR energi e lebih rendah THEN

kunci e sebagai target

ENDIF

END_EVENT
END

4.3 Implementasi Kurayami sebagai Bot Utama

4.3.1 Penyimpanan Data Target

Kurayami menggunakan struktur `TargetInfo` untuk menyimpan informasi musuh. Data yang disimpan meliputi ID, posisi, posisi sebelumnya, energi, arah, kecepatan, jarak, bearing, dan turn terakhir terlihat. Struktur ini cukup sederhana, tetapi sudah cukup untuk mendukung pemilihan target, prediksi tembakan, dan movement.

Informasi posisi dan kecepatan digunakan untuk memperkirakan arah gerak target. Informasi energi digunakan untuk menentukan prioritas target dan power tembakan. Sementara itu, `LastSeenTurn` digunakan agar bot dapat mengetahui apakah data target masih layak dipakai atau sudah terlalu lama.

4.3.2 Pemilihan Target

Pemilihan target dilakukan melalui `ShouldSwitchTarget` dan `TargetScore`. Target baru hanya dipilih jika nilainya lebih baik dari target saat ini. Dengan demikian, bot tidak terlalu sering mengganti target, tetapi tetap dapat berpindah ke target yang lebih menguntungkan.

Pembatasan pergantian target ini penting karena bot yang terlalu sering mengganti target dapat kehilangan fokus. Jika target terus berubah, radar dan gun akan sering diarahkan ulang sehingga peluang tembakan tepat menjadi lebih kecil.

Faktor yang digunakan:

1. Jarak musuh.
2. Energi musuh.
3. Seberapa baru target terlihat radar.

4.3.3 Kontrol Radar

Radar memiliki dua mode. Jika target belum ada, radar melakukan sweep untuk mencari musuh. Jika target sudah ada, radar diarahkan ke posisi target menggunakan mekanisme radar lock. Cara ini membantu bot mempertahankan informasi target dari turn ke turn.

Radar lock juga membantu bot menjaga data target tetap baru. Semakin baru data target, semakin baik keputusan movement dan shooting yang dapat diambil. Jika target terlalu lama tidak terlihat, Kurayami akan menghapus target tersebut dan kembali mencari musuh.

4.3.4 Kontrol Tembakan

Kurayami tidak selalu menembak dengan power besar. Fungsi `ChooseFirePower` menyesuaikan power berdasarkan:

1. Jarak target.
2. Energi target.
3. Energi bot sendiri.
4. Rasio tembakan yang mengenai target.
5. Jumlah peluru yang mengenai dinding.

Jika target jauh atau akurasi sedang buruk, power dikurangi agar energi tidak terbuang. Jika target dekat, power dapat dinaikkan karena peluang terkena lebih besar.

Pendekatan ini merupakan bentuk greedy dalam pengelolaan energi. Pada setiap kesempatan menembak, bot memilih power yang dianggap paling menguntungkan untuk kondisi saat itu. Power tinggi hanya dipakai ketika peluang mengenai target cukup besar atau target berada pada jarak dekat.

4.3.5 Prediksi Posisi Target

Fungsi `PredictTargetPosition` memperkirakan posisi target berdasarkan arah dan kecepatan target. Prediksi ini digunakan terutama saat target berada pada jarak menengah atau jauh. Posisi prediksi juga dibatasi agar tidak keluar dari arena.

Prediksi diperlukan karena peluru membutuhkan waktu untuk sampai ke target. Jika bot hanya menembak posisi target saat ini, tembakan dapat meleset ketika target bergerak cepat. Dengan prediksi sederhana, Kurayami berusaha mengarahkan tembakan ke posisi yang mungkin ditempati target saat peluru tiba.

4.3.6 Pergerakan dan Penghindaran Dinding

Fungsi `MoveByState` menentukan gerakan bot berdasarkan keadaan:

1. Jika target terlalu dekat, bot bergerak menjauh.
2. Jika bot dekat dinding, bot bergerak ke tengah arena.
3. Jika target berada pada jarak aman, bot bergerak lateral terhadap target.

4. Jika belum ada target, bot bergerak patroli sambil radar mencari musuh.

Movement lateral dipilih agar bot tidak bergerak lurus menuju musuh. Gerakan menyamping membuat bot lebih sulit diprediksi oleh lawan.

Selain itu, Kurayami mengubah arah gerak secara periodik atau ketika terkena peluru. Perubahan arah ini membuat pola gerakan tidak terlalu mudah dibaca. Jika bot bergerak dengan pola yang sama terus-menerus, lawan dapat lebih mudah memprediksi posisi bot.

4.4 Pengujian

Pengujian dilakukan dengan format 1 vs 1 antara bot utama Kurayami dan tiga bot alternatif. Bagian hasil dapat diisi setelah pertandingan dijalankan pada engine.

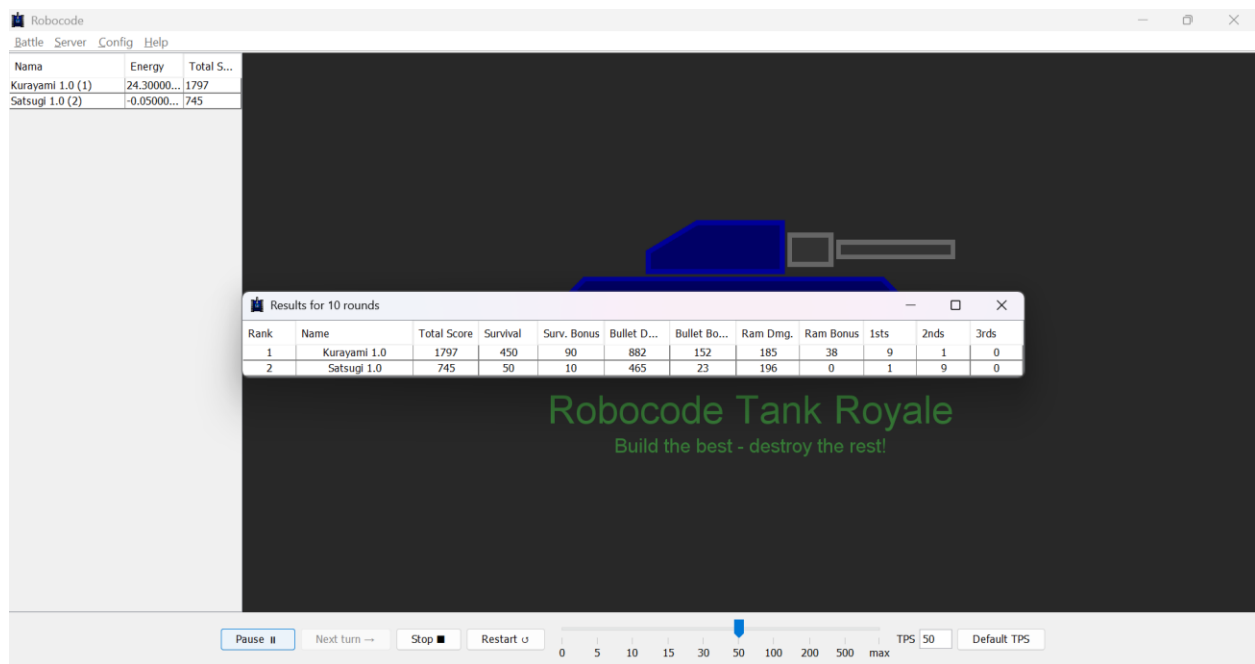
Format 1 vs 1 dipilih agar pengaruh strategi lawan dapat diamati dengan lebih jelas. Dengan hanya dua bot di arena, hasil pertandingan lebih mudah dianalisis karena tidak ada gangguan dari bot ketiga. Setiap pengujian dirancang untuk memperlihatkan respons Kurayami terhadap tipe lawan yang berbeda.

4.4.1 Rencana Pengujian

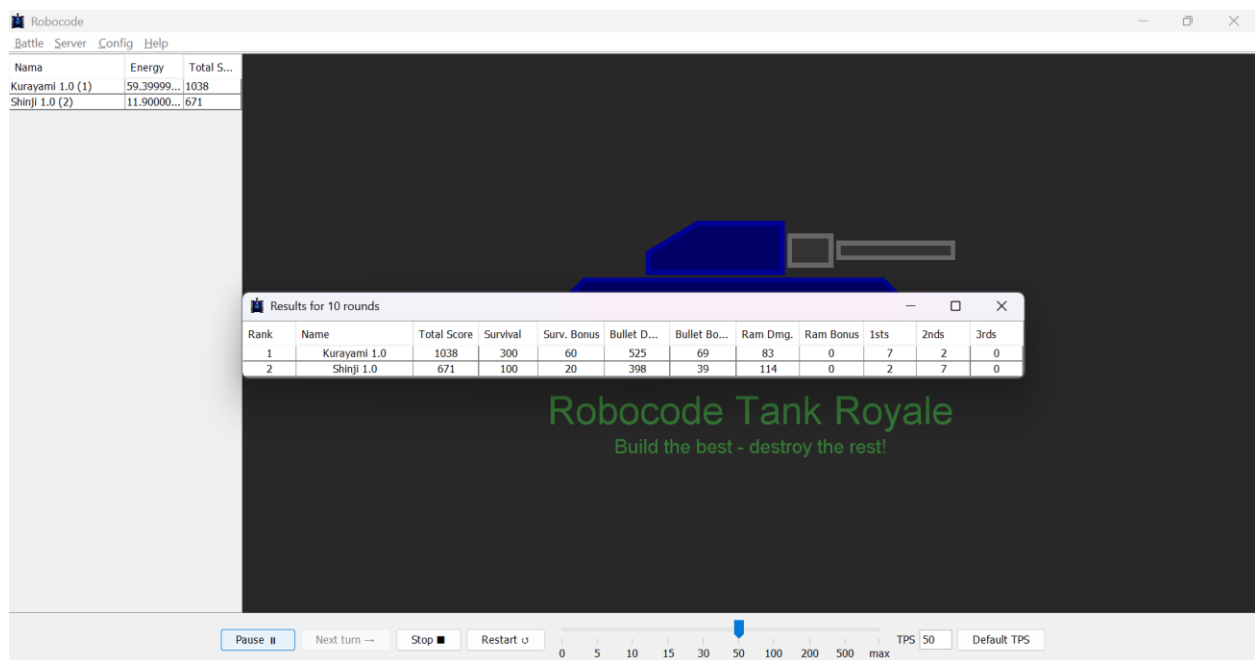
Pengujian	Pertandingan	Tujuan
1	Kurayami vs Satsugi	Menguji Kurayami melawan bot agresif
2	Kurayami vs Shinji	Menguji Kurayami melawan bot defensif
3	Kurayami vs Tatsuya	Menguji Kurayami melawan bot pemburu energi rendah
4	Kurayami vs Satsugi vs Shinji vs Tatsuya	Menguji Kurayami saat semua bot berada dalam satu arena

4.4.2 Hasil Pengujian

Pengujian 1:



Pengujian 2:



Pengujian 3:

Robocode

Battle Server Config Help

Nama	Energy	Total S...
Tatsuya 1.0 (1)	1.421085...	88
Kurayami 1.0 (2)	137.7000...	1632

Results for 10 rounds

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Kurayami 1.0	1722	500	100	863	180	79	0	10	0	0
2	Tatsuya 1.0	88	0	0	88	0	0	0	0	10	0

Robocode Tank Royale
Build the best - destroy the rest!

Pause Next turn Stop Restart

0 5 10 15 30 50 100 200 500 max TPS max Default TPS

Pengujian 4:

Robocode

Battle Server Config Help

Nama	Energy	Total S...
Tatsuya 1.0 (1)	100.0	372
Satsugi 1.0 (2)	-0.19999...	791
Shinji 1.0 (3)	94.40000...	2536
Kurayami 1.0 (4)	112.1999...	2312

Results for 10 rounds

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Shinji 1.0	2536	1000	150	1027	184	175	0	4	4	0
2	Kurayami 1.0	2312	800	60	1142	129	181	0	4	4	0
3	Satsugi 1.0	791	150	0	436	0	157	48	0	0	4
4	Tatsuya 1.0	372	300	0	36	0	36	0	0	0	4

Robocode Tank Royale
Build the best - destroy the rest!

Pause Next turn Stop Restart

0 5 10 15 30 50 100 200 500 max TPS 500 Default TPS

4.4.3 Ringkasan Komponen Skor Penting

Pada pengujian 1 vs 1, Kurayami selalu memperoleh total skor lebih tinggi daripada bot lawan. Melawan Satsugi, Kurayami memperoleh skor 1797, sedangkan Satsugi memperoleh 745. Melawan Shinji, Kurayami memperoleh 1038, sedangkan Shinji memperoleh 671. Melawan Tatsuya, Kurayami memperoleh 1722, sedangkan Tatsuya hanya memperoleh 88.

Pada pengujian semua bot, hasil berubah. Shinji memperoleh peringkat pertama dengan skor 2536, sedangkan Kurayami berada di peringkat kedua dengan skor 2312. Walaupun Kurayami memiliki bullet damage paling tinggi, yaitu 1142, Shinji unggul pada survival score sebesar 1000 dan survival bonus sebesar 150. Hal ini menunjukkan bahwa dalam pertandingan multi-bot, kemampuan bertahan hidup menjadi faktor yang sangat berpengaruh.

4.5 Analisis Pengujian

Analisis pengujian dilakukan dengan melihat apakah Kurayami mampu menjalankan tujuan strateginya. Beberapa indikator yang dapat diamati adalah kemampuan radar mempertahankan target, kemampuan bot menjaga jarak, jumlah tabrakan dinding, penggunaan energi, dan efektivitas tembakan.

Analisis pengujian dilakukan dengan melihat apakah Kurayami mampu menjalankan tujuan strateginya. Beberapa indikator yang diamati adalah total skor, jumlah kemenangan round, bullet damage, survival score, ram damage, dan kemampuan menghadapi tipe lawan yang berbeda.

Selain melihat pemenang, pengujian juga perlu memperhatikan pola pertandingan. Misalnya, bot dapat menang karena damage tinggi, karena bertahan lebih lama, atau karena lawan sering menabrak dinding. Analisis seperti ini penting agar hasil pengujian tidak hanya berupa angka, tetapi juga menjelaskan penyebab performa bot.

Berdasarkan hasil 1 vs 1, Kurayami menunjukkan performa yang baik. Kurayami menang dari Satsugi dengan skor 1797 berbanding 745. Hasil ini menunjukkan bahwa strategi Kurayami cukup efektif menghadapi bot agresif. Satsugi memiliki ram damage lebih tinggi, yaitu 196, tetapi Kurayami unggul pada bullet damage, survival, dan bonus sehingga total skornya lebih besar.

Kurayami diperkirakan bekerja baik ketika target dapat terus dipantau radar dan jarak berada pada zona menengah. Pada kondisi tersebut, bot dapat bergerak lateral dan menembak dengan power yang cukup efisien.

Pada pertandingan melawan Shinji, Kurayami menang dengan skor 1038 berbanding 671. Pertandingan ini lebih ketat dibanding dua pengujian lainnya karena Shinji memiliki strategi defensif dan mampu menjaga jarak. Walaupun demikian, Kurayami tetap unggul karena memperoleh bullet damage lebih tinggi dan lebih sering menjadi peringkat pertama.

Namun, karena menggunakan pendekatan greedy, Kurayami tetap memiliki keterbatasan. Keputusan yang diambil adalah keputusan terbaik menurut kondisi saat itu, bukan keputusan terbaik untuk seluruh round. Akibatnya, Kurayami dapat kurang optimal ketika:

Pada pertandingan melawan Tatsuya, Kurayami menang sangat dominan dengan skor 1722 berbanding 88. Kurayami memperoleh 10 kali peringkat pertama dari 10 round. Hal ini menunjukkan bahwa strategi balanced target greedy milik Kurayami jauh lebih stabil dibanding strategi Tatsuya yang terlalu fokus pada target berenergi rendah.

1. Target bergerak sangat acak.
2. Bot kehilangan target dari radar.
3. Bot terdesak ke dekat dinding.
4. Lawan menyerang sangat agresif dari jarak dekat.
5. Tembakan prediksi sering meleset sehingga energi terbuang.

Hasil berbeda muncul pada pengujian semua bot. Dalam pertandingan Kurayami melawan Satsugi, Shinji, dan Tatsuya sekaligus, Kurayami berada di peringkat kedua dengan skor 2312, sedangkan Shinji menjadi peringkat pertama dengan skor 2536. Kurayami sebenarnya memiliki bullet damage tertinggi, yaitu 1142, tetapi Shinji unggul pada survival score dan survival bonus. Artinya, Kurayami mampu menyerang dengan baik, tetapi dalam kondisi multi-bot, strategi Shinji yang lebih defensif lebih menguntungkan untuk bertahan hidup dan mengumpulkan poin survival.

Dari hasil tersebut, dapat disimpulkan bahwa strategi greedy Kurayami efektif untuk pertandingan 1 vs 1, terutama karena bot mampu memilih target, menjaga jarak, dan menembak dengan cukup konsisten. Namun, pada pertandingan multi-bot, Kurayami masih perlu peningkatan dalam hal pemilihan posisi dan prioritas survival. Keputusan greedy yang terlalu fokus pada target aktif dapat membuat bot kurang memperhatikan risiko dari bot lain di arena.

Keterbatasan Kurayami berdasarkan hasil pengujian adalah:

1. Pada mode multi-bot, Kurayami belum sekuat Shinji dalam mempertahankan survival score.
2. Kurayami masih perlu menyeimbangkan antara mengejar bullet damage dan bertahan hidup.
3. Target selection perlu mempertimbangkan ancaman dari beberapa bot, bukan hanya target yang sedang dikunci.
4. Movement perlu lebih adaptif agar bot tidak mudah masuk ke posisi berisiko saat banyak bot aktif.
5. Strategi khusus untuk 1 vs 1 dan multi-bot perlu dipisahkan agar keputusan greedy lebih sesuai dengan kondisi pertandingan.

5.1 Kesimpulan

Berdasarkan implementasi yang telah dibuat, algoritma greedy dapat diterapkan dalam Robocode Tank Royale untuk mengatur pemilihan target, pergerakan, radar, dan penembakan. Setiap bot pada proyek ini menggunakan heuristic yang berbeda sehingga dapat dibandingkan karakter dan efektivitasnya.

Berdasarkan implementasi dan pengujian yang telah dilakukan, algoritma greedy dapat diterapkan dalam Robocode Tank Royale untuk mengatur pemilihan target, pergerakan, radar, dan penembakan. Setiap bot pada proyek ini menggunakan heuristic yang berbeda sehingga karakter strategi masing-masing dapat dibandingkan melalui pertandingan.

Kurayami sebagai bot utama berhasil menunjukkan performa baik pada pengujian 1 vs 1. Kurayami menang melawan Satsugi, Shinji, dan Tatsuya. Hasil terbaik terlihat saat melawan Tatsuya, yaitu Kurayami memperoleh skor 1722 dan menang pada seluruh round. Hal ini menunjukkan bahwa strategi Kurayami yang seimbang lebih stabil dibanding strategi yang hanya berfokus pada satu heuristic.

Kurayami dipilih sebagai bot utama karena memiliki pendekatan yang paling seimbang. Bot ini mempertimbangkan jarak, energi musuh, usia informasi target, energi sendiri, akurasi, dan posisi terhadap dinding. Dengan kombinasi tersebut, Kurayami tidak hanya mengejar damage, tetapi juga menjaga peluang bertahan hidup.

Pada pertandingan semua bot, Kurayami belum menjadi peringkat pertama. Kurayami memperoleh skor 2312 dan berada di bawah Shinji yang memperoleh skor 2536. Meskipun Kurayami unggul dalam bullet damage, Shinji memperoleh survival score dan survival bonus yang lebih tinggi. Dari hasil ini dapat disimpulkan bahwa strategi Kurayami kuat untuk duel, tetapi masih perlu ditingkatkan untuk kondisi multi-bot.

Secara umum, pendekatan greedy membantu menyederhanakan pengambilan keputusan bot. Bot tidak perlu menghitung seluruh kemungkinan pertandingan, tetapi cukup memilih aksi terbaik berdasarkan informasi yang tersedia. Walaupun tidak selalu optimal secara global, strategi ini cukup sesuai untuk permainan real-time seperti Robocode Tank Royale.

1. Membuat mode khusus untuk 1 vs 1 dan melee.
2. Meningkatkan prediksi tembakan terhadap musuh yang bergerak zig-zag.
3. Menambahkan strategi menghindari peluru.
4. Melakukan tuning parameter jarak, fire power, dan radar berdasarkan hasil pengujian yang lebih banyak.
5. Menambahkan pertimbangan survival yang lebih kuat pada mode multi-bot.
6. Melakukan tuning parameter jarak, fire power, dan radar berdasarkan hasil pengujian yang lebih banyak.

Saran tersebut bertujuan agar bot tidak hanya reaktif terhadap target, tetapi juga semakin adaptif terhadap gaya bermain lawan. Dengan pengujian lebih banyak, nilai parameter seperti jarak aman, batas target hilang, dan power tembakan dapat disesuaikan agar performa bot menjadi lebih stabil.

Saran tersebut bertujuan agar bot tidak hanya reaktif terhadap target, tetapi juga semakin adaptif terhadap gaya bermain lawan. Berdasarkan hasil pengujian, peningkatan paling penting adalah membuat Kurayami lebih kuat dalam pertandingan multi-bot. Dengan pengujian lebih banyak, nilai parameter seperti jarak aman, batas target hilang, dan power tembakan dapat disesuaikan agar performa bot menjadi lebih stabil.

Lampiran

A. Tautan

Repository GitHub: https://github.com/crismanturnip/Tubes_Tantrump

Video demo:

<https://drive.google.com/file/d/1q06DaMft3nYLteYovb2vhRXNEIEcsnt/view?usp=sharing>

B. Checklist Pengerjaan

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada engine Robocode Tank Royale	✓	
2	Membuat 4 solusi greedy dengan heuristic berbeda	✓	
3	Membuat laporan sesuai spesifikasi	✓	
4	Melakukan pengujian minimal 3 kali format 1 vs 1	✓	
5	Menyertakan tautan GitHub	✓	
6	Menyertakan video demo jika ada	✓	

Daftar Pustaka

Robocode Developer. (n.d.). *Robocode Tank Royale Bot API Documentation*. <https://robocode-dev.github.io/tank-royale/api/dotnet/api/Robocode.TankRoyale.BotApi.html>

Robocode Developer. (n.d.). *Robocode Tank Royale*. <https://robocode-dev.github.io/tank-royale/>

Munir, R. (n.d.). *Strategi Algoritma: Algoritma Greedy*. Institut Teknologi Bandung.

Wikipedia contributors. (n.d.). *Greedy algorithm*. Wikipedia. https://en.wikipedia.org/wiki/Greedy_algorithm

Tim TubesStima. (2026). *Kode program bot Kurayami, Satsugi, Shinji, dan Tatsuya pada proyek TubesStima*.